



bruteforce & recaptcha

1. docker 우분투 container 생성

```
docker run -it --name brute-ubuntu ubuntu
```

2. 필요한 패키지 설치

```
apt update
```

```
apt install -y vim
```

```
apt install -y curl
```

3. 테스트 코드 작성

```
vi test.sh
```

```
for i in $(seq 1 100); do  
    password="pass$i"  
    echo "Trying: $password"  
  
    res=$(curl -s -X POST http://host.docker.internal:8087/api/login \  
        -H "Content-Type: application/json" \  
        -d '{"password": "'$password'"}')
```

```
-d "{\"username\":\"admin\", \"password\":\"$password\"}")

echo "$res" | grep -Eq "Invalid credentials|reCAPTCHA failed"
if [ $? -ne 0 ]; then
    echo "[+] Password found: $password"
    break
fi
done
```

```
chmod 777 ./test.sh
```

4. 테스트 서버 코드 생성

[LoginController.java]

```
package com.example.monitoring.controller;

import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;

@RestController
public class LoginController {

    @PostMapping("/api/login")
    public String postMethodName(@RequestBody Map<String, Object> entity) {
        System.out.println(">>>>> test");

        return "Invalid credentials";
    }
}
```

```
}
```

5. 테스트 코드실행

```
./test.sh
```

100번의 요청이 진행되고

1~100을 비밀번호로 넣는 brute force 테스트다.

[illegible]

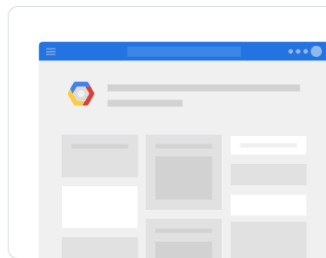
요청이 계속 오는 걸 확인할 수 있다.

디도스처럼 수많은 요청을
막기 위해
recapcha를 이용해보자.

6. recaptcha 셋팅

<https://www.google.com/recaptcha/admin>

← 새 사이트 등록



reCAPTCHA 시작하기

MFA, 스텝/사기 방지, Google Cloud 통합과 같은 고급 기능도 지원됩니다.

- ✓ 무료 월간 평가 최대 10,000회
- ✓ 신용카드는 필요하지 않습니다.

라벨

test

4/50

reCAPTCHA 유형

- ☐ 점수 기준(v3) 점수를 사용하여 요청을 확인합니다.
- ☒ 테스트(v2) 보안문자를 사용하여 요청을 확인합니다.
 - ☒ "로봇이 아닙니다." 체크박스 "로봇이 아닙니다." 체크박스를 사용하여 요청을 확인합니다.
 - ☐ 표시되지 않는 reCAPTCHA 배지 백그라운드에서 요청을 확인합니다.

도메인

+ localhost

Google Cloud Platform

프로젝트 이름*

springboot-developer

20/30

이전에 Google Cloud를 사용하신 것으로 보입니다. 시작하기 위해 새 프로젝트를 만들고 필요한 API를 사용 설정 하겠습니다.

GOOGLE CLOUD PLATFORM

취소

제출

생성한 뒤

사이트키, 비밀키를 확인하자



설정이 완료됨

- ✓ Google Cloud 프로젝트의 설정 관리
- ✓ 무료 월간 평가 최대 10,000회

reCAPTCHA 키를 호스팅하는 [Google Cloud Platform 프로젝트](#)로 이동하여 고급 기능을 사용 설정하세요.

사이트에서 사용자에게 제공하는 HTML 코드에 이 사이트 키를 사용하세요. [클라이언트 측 통합 자세히 알아보기](#)

🔑 사이트 키 복사

1000

사이트와 reCAPTCHA의 커뮤니케이션을 위해 이 비밀 키를 사용하세요. [서버 측 통합 자세히 알아보기](#)

🔑 비밀 키 복사

[설정으로 이동](#)

애널리틱스로 이동

7. 프론트, 백 소스 추가

[recapcha.html]

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script src="https://www.google.com/recaptcha/api.js" async defer></script>
</head>
```

```

<body>
  <div>
    <div class="g-recaptcha" data-sitekey="본인엑세스키" data-callback="on
    CaptchaSuccess"
    data-expired-callback="onCaptchaExpired">
    </div>

  </div>
  <div class="login" id="loginform" style="display: none;">
    <input type="text" id="username">
    <input type="password" id="password">
    <button id="goLogin">Login</button>
  </div>
</div>
</body>

</html>

```

```

<script>

```

```

function onCaptchaSuccess(token) {
  console.log('token > ', token);
  window.token = token;
  document.getElementById('loginform').style.display = 'block';
}

```

```

function onCaptchaExpired() {
  document.getElementById('loginform').style.display = 'none';
}

```

```

const loginBtn = document.querySelector('#goLogin');
loginBtn.addEventListener('click', goLogin);

```

```

function goLogin() {
  fetch('http://localhost:8080/api/login', {
    method: 'POST',

```

```

headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({
  username: 'admin',
  password: '1234',
  recaptchaToken: window.token
})
}).then(res => res.text()).then(console.log);
}

</script>

```

[LoginController.java]

```

package com.example.monitoring.controller;

import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.client.RestTemplate;

import java.util.Map;

import org.springframework.http.HttpEntity;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.util.LinkedMultiValueMap;
import org.springframework.util.MultiValueMap;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;

@RestController
@CrossOrigin("*")
public class LoginController {

    private final String RECAPTCHA_SECRET = "본인비밀키";

    private boolean verifyRecaptchaToken(String token) {

```



```

String verifyUrl = "https://www.google.com/recaptcha/api/siteverify";

RestTemplate restTemplate = new RestTemplate();
MultiValueMap<String, String> body = new LinkedMultiValueMap<>();
body.add("secret", RECAPTCHA_SECRET);
body.add("response", token);

HttpHeaders headers = new HttpHeaders();
headers.setContentType(MediaType.APPLICATION_FORM_URLENCODED);

HttpEntity<MultiValueMap<String, String>> request = new HttpEntity<>(body, headers);

try {
    ResponseEntity<Map> response = restTemplate.postForEntity(verifyUrl, request, Map.class);
    Map<String, Object> responseBody = response.getBody();
    return (Boolean) responseBody.get("success");
} catch (Exception e) {
    e.printStackTrace();
    return false;
}

@PostMapping("/api/login")
public ResponseEntity<?> postMethodName(@RequestBody Map<String, Object> entity) {
    System.out.println(">>>>> entity " + entity);

    boolean verified = verifyRecaptchaToken((String) entity.get("recaptchaToken"));
    if (!verified) {
        return ResponseEntity.badRequest().body("reCAPTCHA failed");
    }

    // 로그인 처리를 여기에다가 하면 됨..

    return ResponseEntity.ok("Login success");
}

```

}

}



1. checkbox 선택후 recaptcha 서버에 인증 확인
2. recaptcha 서버에서 토큰 발급해줌
3. 발급한 토큰과 함께 로그인 정보를 우리의 백엔드로 전송
4. 백엔드에서 토큰이 유효한지 recaptcha 서버에 확인 요청

- 정상이면 로그인 로직 수행
 - 비정상이면 에러메세지 리턴
-